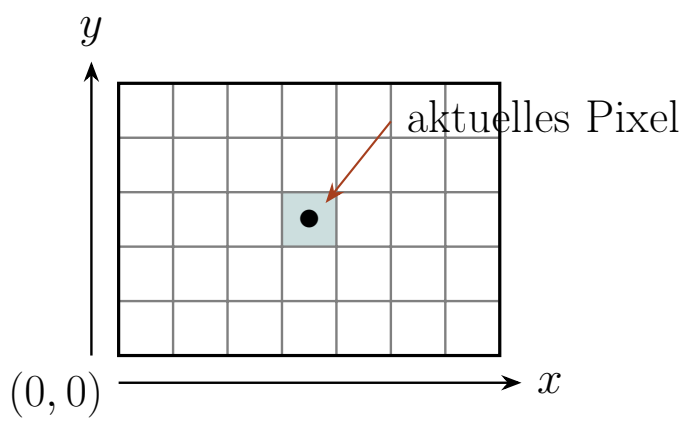
1 · Ein Bild aus Pixeln



Für jedes Pixel wird die Farbe berechnet:

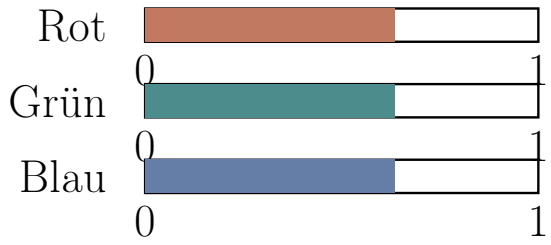
$$\boxed{\text{Farbe} = f(x, y, t)}$$

`fragCoord`	Position des aktuellen Pixels
`iResolution.xy`	Bildbreite und Bildhöhe
`iTime`	Zeit in Sekunden

`mainImage` berechnet eine Farbe pro Pixel.

Live aufbauen: Raster zeichnen → ein Pixel markieren → Position, Bildgröße und Zeit benennen → Farbformel.

2 · Farbe und Zeit



$$\text{Farbe} = (R, G, B)$$

Jeder Wert liegt zwischen 0 und 1.
$(0, 0, 0)$ ist Schwarz, $(1, 1, 1)$ ist Weiß.

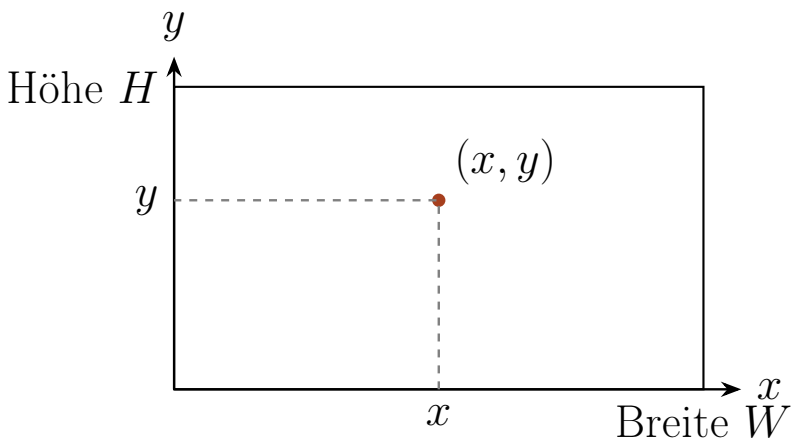
Mit Zeit ändert sich die Farbe überall zugleich:

$$R(t) = 0{,}5 + 0{,}5 \sin(t)$$

Grün und Blau schwingen zeitversetzt.

Live aufbauen: RGB-Anteile eintragen → zwei Farben vergleichen → Zeit als Eingabe ergänzen → Wertebereich des Sinus erklären.

3 · Position wird Farbe



$$u = \frac{x}{W}, \qquad v = \frac{y}{H}$$

Positionsfarbe: $(R, G, B) = (u, v, 0)$.

Zwei Farben entlang der x-Achse mischen:

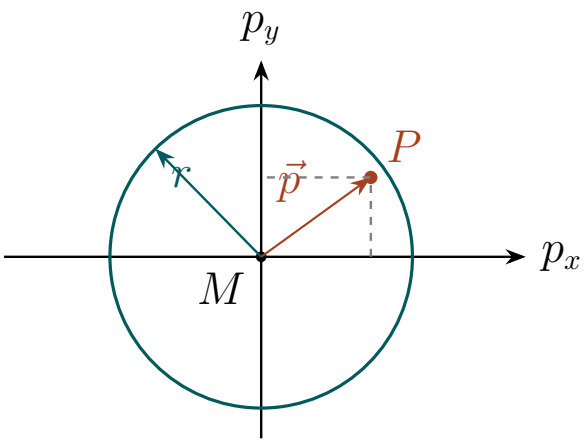
$$\text{Farbe} = (1 - u)\,\text{links} + u\,\text{rechts}$$

$u = 0$: links; $\quad u = 1$: rechts.

Getrennt normierte Achsen sind für Abstände noch nicht gleich skaliert.

Live aufbauen: Pixel und Auflösung markieren → normieren → Koordinaten als Farben zeigen → zwei Farben mischen.

4 · Vom Abstand zum Kreis



Bildmitte als Ursprung, durch Bildhöhe skalieren:

$$\vec{p} = \frac{\texttt{fragCoord} - \frac{1}{2}\,\texttt{iResolution.xy}}{\texttt{iResolution.y}}$$

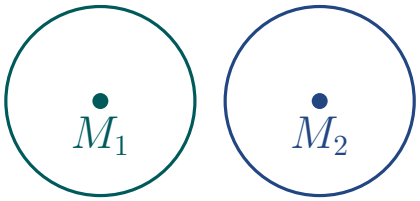
Gleiche Einheiten auf beiden Achsen halten den Kreis rund.

$$d^2 = p_x^2 + p_y^2 = \mathrm{dot}(\vec{p}, \vec{p})$$

innen: $d^2 < r^2$ außen: $d^2 \geq r^2$

Live aufbauen: Mittelpunkt und Pixel einzeichnen → Koordinaten zentrieren → Abstand → Radiusvergleich und zwei Farben.

5 · Von Kreisen zu Metaballs



Zwei Felder beeinflussen jeden Punkt.

Bewegung der Mittelpunkte mit t

Ein Feld wird mit wachsendem Abstand kleiner:

$$E_i(P) = \frac{a}{\|P - M_i\|^2 + \varepsilon}$$

Die Felder addieren sich:

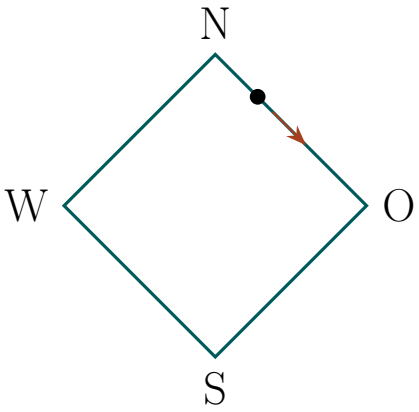
$$E(P) = E_1(P) + E_2(P)$$

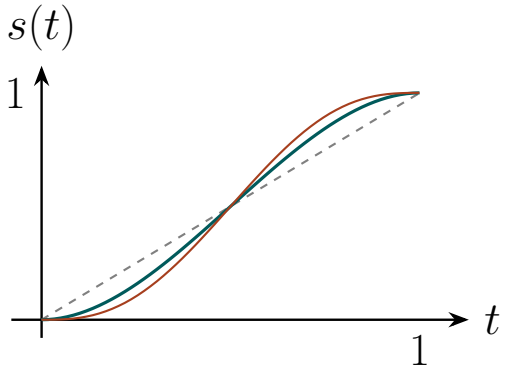
$$E(P) > \text{Schwelle} \quad \Rightarrow \quad \text{Vordergrund}$$

Zwischen den Zentren kann die Summe die Schwelle überschreiten: Die Formen verbinden sich. ε verhindert Division durch null.

Live aufbauen: Kreis → ein abfallendes Feld und Schwelle → zweites Feld addieren → Bewegung.

6 · Sanft bremsen und anfahren





Zwischen zwei benachbarten Ecken:

$$P(t) = (1 - s(t))A + s(t)B$$

Lerp: lineare Interpolation, in GLSL `mix`.
Je Kante: 1,2 s Fahrt + 0,6 s Halt.
`clamp` hält den Zeitfortschritt am Ende bei 1.
Im Halt: Schwarz → Eckfarbe → Schwarz.
N: Rot; O: Grün; S: Blau; W: Gelb.
Farbimpuls $u = 1 - |2h - 1|$ für Haltefortschritt h.
Dieselbe Kurve s mischt die Farbe:

$$C(h) = s(u)\, C_{\text{Ecke}}$$

Linear: $s(t) = t$; abrupter Start und Halt.

Smoothstep: $s(t) = 3t^2 - 2t^3$.
Aus $s(0) = 0$, $s(1) = 1$, $s'(0) = s'(1) = 0$ mit einem kubischen Ansatz herleiten.

Smootherstep: $s(t) = 6t^5 - 15t^4 + 10t^3$.
Zusätzlich gilt $s''(0) = s''(1) = 0$; auch die Beschleunigung geht an den Enden auf null.

Die Steigung des Graphen bestimmt die Geschwindigkeit entlang der Kante. Die Bahn bleibt dieselbe.

Kanon ab Phase 2: zwei, später vier Punkte mit lokaler Zeit

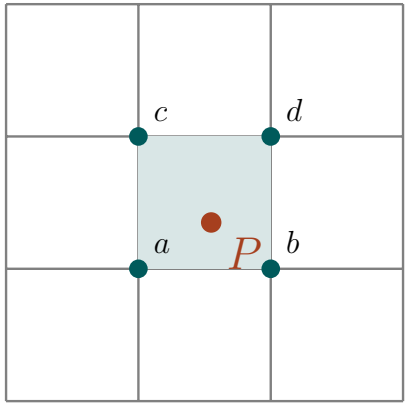
$$T_j = (T - 1{,}5j) \bmod 7{,}2.$$

Phase 2/3: $j = 0, 2$ (Stimmen 1 und 3).

Ab Phase 4: $j = 0, 1, 2, 3$ (alle vier Stimmen).

Live aufbauen: Ein Punkt mit LERP und Farbwechsel → Schleifengrenze 1 auf 4: Stimme 3 ergänzen → Smoothstep herleiten und einsetzen → Stimmen 2 und 4 ergänzen → Smootherstep einsetzen und vergleichen.

7 · Vom Zufall zu Noise



Vier feste Zufallswerte a, b, c, d umgeben das Pixel P.

$$u = \mathrm{mix}(a, b, s(t_x))$$
$$v = \mathrm{mix}(c, d, s(t_x))$$
$$N = \mathrm{mix}(u, v, s(t_y))$$

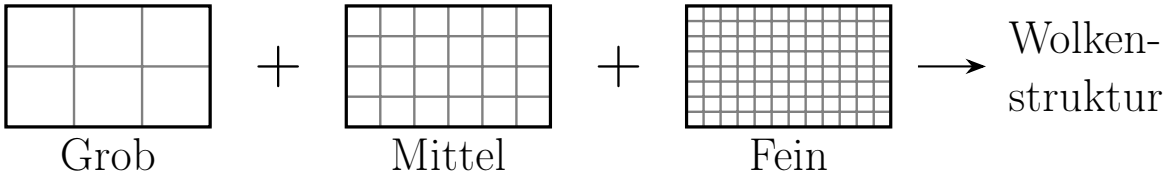
1. **UV-Bild:** normierte Positionen als Rot und Grün.
2. **Pixelzufall:** ein fester Grauwert je Pixel; keine Zeitabhängigkeit.
3. **Value Noise:** Zufallswerte am gröberen Gitter verankern und dazwischen mischen.
4. **Simplex Noise:** vorbereitete Alternative; gewichtete Gradientenbeiträge auf einem Dreiecksgitter.

Bei Value Noise: floor(p) wählt die Zelle; fract(p) liefert (t_x, t_y) darin. Das bekannte Polynom $s(t) = 3t^2 - 2t^3$ glättet die Mischung.

Simplex interpoliert keine skalaren Eckwerte. Die Basis wird übernommen; `simplexNoise01` rechnet intern auf $[0, 1]$ um. Im Gestaltungs-Code wechselt nur der Funktionsname.

Live aufbauen: UV wiedererkennen → unabhängige Pixelwerte → Gitter und vier Eckwerte → mit bekanntem Polynom mischen → Simplex als alternative Basis einordnen.

8 · Aus Noise werden bewegte Wolken



$$F(p) = \frac{0{,}5N(p) + 0{,}25N(2p) + 0{,}125N(4p) + 0{,}0625N(8p)}{0{,}9375}$$

1. **START:** N ist das vorbereitete `simplexNoise01` aus Demo 5, mit Werten in $[0, 1]$.
2. **fBm:** vier Oktaven; Frequenz verdoppeln, Gewicht halbieren. Das Bild zeigt drei Maßstäbe.
3. **Wolken:** $d = \text{smoothstep}(0{,}4;\ 0{,}65;\ F(p))$; Farbe $= \text{mix}(\text{Blau}, \text{Weiß}, d)$.
4. **Animation:** $p \leftarrow p - (0{,}18;\ 0{,}06)\, t$. Das vorhandene Muster wandert.

Live aufbauen: Simplex-Ziel übernehmen → gewichtete Oktaven ergänzen → Dichte und Farbe gestalten → mit der Zeit Abfrageposition verschieben.